

```

1 #Modulo de llamadas http para descargar ficheros
2 !pip install requests
3
4 #Libreria del problema TSP: http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib.html
5 !pip install tsplib95

```

```

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.2.3)
Requirement already satisfied: certifi<=2024.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2024.4.17)
Collecting tsplib95
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Click<=6.0 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (8.3.1)
Collecting Deprecated<=1.2.9 (from tsplib95)
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Collecting networkx<=2.1 (from tsplib95)
  Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
Collecting tabulate<=0.8.7 (from tsplib95)
  Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
Collecting wrapt<2,>=1.10 (from Deprecated<=1.2.9->tsplib95)
  Downloading wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl.metadata (88 kB)
Download tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
Download Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
Download networkx-2.8.8-py3-none-any.whl (2.0 MB)
----- 2.0/2.0 MB 53.9 MB/s eta 0:00:00
Download tabulate-0.8.10-py3-none-any.whl (29 kB)
Download wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl (88 kB)
----- 88.0/88.0 kB 6.4 MB/s eta 0:00:00
Installing collected packages: wrapt, tabulate, networkx, Deprecated, tsplib95
  Attempting uninstall: wrapt
    Found existing installation: wrapt 2.1.1
    Uninstalling wrapt-2.1.1:
      Successfully uninstalled wrapt-2.1.1
  Attempting uninstall: tabulate
    Found existing installation: tabulate 0.9.0
    Uninstalling tabulate-0.9.0:
      Successfully uninstalled tabulate-0.9.0
  Attempting uninstall: networkx
    Found existing installation: networkx 3.6.1
    Uninstalling networkx-3.6.1:
      Successfully uninstalled networkx-3.6.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This is
bigframes 2.36.0 requires tabulate<=0.9, but you have tabulate 0.8.10 which is incompatible.
scikit-image 0.25.2 requires networkx<=3.0, but you have networkx 2.8.8 which is incompatible.
mapclassify 2.10.0 requires networkx<=3.2, but you have networkx 2.8.8 which is incompatible.
momepy 0.11.0 requires networkx<=3.2, but you have networkx 2.8.8 which is incompatible.
spopt 0.7.0 requires networkx<=3.2, but you have networkx 2.8.8 which is incompatible.
Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8.10 tsplib95-0.7.1 wrapt-1.17.3

```

```

1 import tsplib95
2 import random
3 from math import e
4 import urllib.request

```

```

1 #DATOS DEL PROBLEMA
2 # file = "swiss42.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/t
3 # !gzip -d swiss42.tsp.gz      #Descomprimir el fichero de datos
4
5 # Estaba caido el server, por lo que lo hacemos usando esta URL:
6 file = "swiss42.tsp" ;
7 urllib.request.urlretrieve("https://raw.githubusercontent.com/coin-or/jorlib/refs/heads/master/jorlib-core/s
8
9 problem = tsplib95.load(file)
10
11 #Nodos
12 Nodos = list(problem.get_nodes())
13
14 #Devuelve la distancia entre dos nodos
15 def distancia(a,b, problem):
16     return problem.get_weight(a,b)
17
18 #Devuelve la distancia total de una trayectoria/solucion(lista de nodos)
19 def distancia_total(solucion, problem):
20     distancia_total = 0
21     for i in range(len(solucion)-1):
22         distancia_total += distancia(solucion[i],solucion[i+1] , problem)
23     return distancia_total + distancia(solucion[len(solucion)-1],solucion[0], problem)
24

```

Algoritmo de colonia de hormigas

La función Add_Nodo selecciona al azar un nodo con probabilidad uniforme. Para ser mas eficiente debería seleccionar el próximo nodo siguiendo la probabilidad correspondiente a la ecuación:

$$p_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in J_i^k} [\tau_{il}(t)]^\alpha [\eta_{il}]^\beta}, \text{ si } j \in J_i^k$$

$$p_{ij}^k(t) = 0, \text{ si } j \notin J_i^k$$

```

1 def Add_Nodo(problem, H ,T ) :
2     #Mejora:Establecer una funcion de probabilidad para
3     # añadir un nuevo nodo dependiendo de los nodos mas cercanos y de las feromonas depositadas
4     Nodos = list(problem.get_nodes())
5     return random.choice( list(set(range(1,len(Nodos))) - set(H) ) )
6
7
8 def Incrementa_Feromona(problem, T, H ) :
9     #Incrementa segun la calidad de la solución. Añadir una cantidad inversamente proporcional a la distancia
10    for i in range(len(H)-1):
11        T[H[i]][H[i+1]] += 1000/distancia_total(H, problem)
12    return T
13
14 def Evaporar_Feromonas(T) :
15    #Evapora 0.3 el valor de la feromona, sin que baje de 1
16    #Mejora:Podemos elegir diferentes funciones de evaporación dependiendo de la cantidad actual y de la suma
17    T = [[ max(T[i][j] - 0.3 , 1) for i in range(len(Nodos)) ] for j in range(len(Nodos))]
18    return T

```

```

1 def hormigas(problem, N) :
2     #problem = datos del problema
3     #N = Número de agentes(hormigas)
4
5     #Nodos
6     Nodos = list(problem.get_nodes())
7     #Aristas
8     Aristas = list(problem.get_edges())
9
10    #Inicializa las aristas con una cantidad inicial de feromonas:1
11    #Mejora: inicializar con valores diferentes dependiendo diferentes criterios
12    T = [[ 1 for _ in range(len(Nodos)) ] for _ in range(len(Nodos))]
13
14    #Se generan los agentes(hormigas) que serán estructuras de caminos desde 0
15    Hormiga = [[0] for _ in range(N)]
16
17    #Recorre cada agente construyendo la solución
18    for h in range(N) :
19        #Para cada agente se construye un camino
20        for i in range(len(Nodos)-1) :
21
22            #Elige el siguiente nodo
23            Nuevo_Nodo = Add_Nodo(problem, Hormiga[h] ,T )
24            Hormiga[h].append(Nuevo_Nodo)
25
26            #Incrementa feromonas en esa arista
27            T = Incrementa_Feromona(problem, T, Hormiga[h] )
28            #print("Feromonas(1)", T)
29
30            #Evapora Feromonas
31            T = Evaporar_Feromonas(T)
32            #print("Feromonas(2)", T)
33
34            #Seleccionamos el mejor agente
35            mejor_solucion = []
36            mejor_distancia = 10e100
37            for h in range(N) :
38                distancia_actual = distancia_total(Hormiga[h], problem)
39                if distancia_actual < mejor_distancia:
40                    mejor_solucion = Hormiga[h]
41                    mejor_distancia =distancia_actual
42
43            print(mejor_solucion)
44            print(mejor_distancia)
45
46
47
48    hormigas(problem, 1000)

```

```

[0, 32, 8, 38, 9, 41, 27, 4, 28, 33, 40, 39, 22, 29, 21, 6, 23, 24, 10, 1, 18, 16, 15, 26, 13, 37, 20, 36, 34, 11
3877

```

